# Práctica N°3: Circuitos Secuenciales Regulares

## Índice


**1. Objetivos** **2**
- 1.1. Objetivos Generales . . . 2
- 1.2. Objetivos Específicos . . . 2

**2. Descripción del Circuito en VHDL** **2**
- 2.1. Modulo 1: Circuito PWM . . . 2
- 2.2. Modulo 2: Circuito Contador . . . 7
  - 2.2.1. Código VHDL Completo del Contador . . . 7
  - 2.2.2. Definición de la Entidad y Parámetro Genérico . . . 8
  - 2.2.3. Señales Internas . . . 8
  - 2.2.4. Proceso de Conteo . . . 9
- 2.3. Modulo 3: Circuito Reset . . . 9
  - 2.3.1. Código VHDL Completo del Circuito Reset . . . 10
  - 2.3.2. Definición de la Entidad . . . 11
  - 2.3.3. Señales Internas . . . 11
  - 2.3.4. Proceso de Registro de Desplazamiento . . . 11
  - 2.3.5. Proceso de Generación de la Señal Reset Interna . . . 12
- 2.4. Archivo principal . . . 12
  - 2.4.1. Código VHDL completo del nivel superior . . . 13
  - 2.4.2. Análisis de parámetros y diseño . . . 15

**3. Diagrama de Bloques** **16**
- 3.1. Diagrama General del Módulo Principal . . . 16
- 3.2. Módulo RESET . . . 17
- 3.3. Módulo COUNTER . . . 17
- 3.4. Módulo PWM . . . 18

**4. Asignación de Pines en la Nexys A7-100T** **18**

**5. Resultados en la Placa Nexys A7 100T** **19**

# 1. Objetivos

## 1.1. Objetivos Generales

1. Implementar un controlador de modulación por ancho de pulso (PWM) para el control de la potencia luminosa de un LED, aplicando técnicas de descripción de hardware.

## 1.2. Objetivos Específicos

1. Diseñar el circuito digital capaz de generar una señal PWM con ciclo de trabajo ajustable.

2. Describir el circuito en un lenguaje de descripción de hardware (HDL) siguiendo las buenas prácticas de diseño.

3. Simular el funcionamiento del controlador PWM para verificar el ajuste del brillo del LED.

4. Implementar y probar el diseño en la tarjeta de desarrollo, validando el control de brillo en condiciones reales.

# 2. Descripción del Circuito en VHDL

El diseño se ha dividido en tres módulos principales: **circuito PWM**, **Circuito Contador** y **Circuito Reset**. En esta sección se describe en detalle el primer bloque, encargado de generar la señal PWM (Pulse Width Modulation) cuyo ciclo de trabajo es ajustable mediante la entrada `duty_cycle`. Este circuito se utiliza para controlar la potencia entregada a una carga, en este caso, un LED, variando el tiempo en que la señal permanece en alto durante cada periodo.

## 2.1. Modulo 1: Circuito PWM

Primero se presenta el código completo del modulo para posteriormente analizar cada parámetro relevantes.

```vhdl
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity pwm is
  generic (
    pwm_bits : integer;
    clk_cnt_len : positive := 1
  );
  port (
    clk         : in std_logic;
    rst         : in std_logic;
    duty_cycle : in unsigned (pwm_bits - 1 downto 0);
    pwm_out     : out std_logic
  );
end pwm;

architecture rtl of pwm is
  signal pwm_cnt : unsigned (pwm_bits - 1 downto 0);
  signal clk_cnt : integer range 0 to clk_cnt_len - 1;
begin
  PWM_PROC : process( clk )
  begin
    if rising_edge(clk) then
      if rst = '1' then
        pwm_cnt <= (others => '0');
        pwm_out <= '0';
      else
        if clk_cnt_len = 1 or clk_cnt = 0 then
          pwm_cnt <= pwm_cnt + 1; -- contador infinito
          pwm_out <= '0';

          if pwm_cnt = unsigned(to_signed(-2, pwm_cnt'
             length)) then
            pwm_cnt <= (others => '0');
          end if;

          if pwm_cnt < duty_cycle then
            pwm_out <= '1';
          end if;
        end if;
      end if;
```

```
42      end if;
43    end process;
44
45    CLK_CNT_PROC : process( clk )
46    begin
47      if rising_edge(clk) then
48        if rst = '1' then
49          clk_cnt <= 0;
50        else
51          if clk_cnt < clk_cnt_len - 1 then
52            clk_cnt <= clk_cnt + 1;
53          else
54            clk_cnt <= 0;
55          end if;
56        end if;
57      end if;
58    end process;
59  end architecture;
```

Listing 1: Código completo del módulo PWM

### Definición de la Entidad y Parámetros

En primer lugar, se declara la entidad `pwm`, la cual incluye parámetros genéricos para definir la resolución de la señal y la división de frecuencia:

```
1  library ieee;
2  use ieee.std_logic_1164.all;
3  use ieee.numeric_std.all;
4
5  entity pwm is
6    generic (
7      pwm_bits : integer;
8      clk_cnt_len : positive := 1
9    );
10   port (
11     clk         : in std_logic;
12     rst         : in std_logic;
13     duty_cycle : in unsigned (pwm_bits - 1 downto 0);
14     pwm_out     : out std_logic
15   );
16 end pwm;
```

Listing 2: Entidad del módulo PWM

El parámetro `pwm_bits` define el número de bits del contador principal, lo que determina la resolución del PWM. El parámetro `clk_cnt_len` permite implementar un prescaler para reducir la frecuencia de la señal de reloj utilizada en la generación del PWM.

### Arquitectura y Señales Internas

Dentro de la arquitectura `rtl`, se definen dos señales internas:

- `pwm_cnt`: contador principal que determina el ciclo de trabajo.
- `clk_cnt`: contador auxiliar para dividir la frecuencia del reloj.

```
architecture rtl of pwm is
  signal pwm_cnt : unsigned (pwm_bits - 1 downto 0);
  signal clk_cnt : integer range 0 to clk_cnt_len - 1;
```

Listing 3: Definición de señales internas

### Proceso de Generación de PWM

El proceso `PWM_PROC` es el encargado de incrementar el contador principal y generar la señal PWM. Cuando `pwm_cnt` es menor que `duty_cycle`, la salida se coloca en alto, generando el pulso correspondiente.

```
PWM_PROC : process( clk )
begin
  if rising_edge(clk) then
    if rst = '1' then
      pwm_cnt <= (others => '0');
      pwm_out <= '0';
    else
      if clk_cnt_len = 1 or clk_cnt = 0 then
        pwm_cnt <= pwm_cnt + 1;
        pwm_out <= '0';

        if pwm_cnt = unsigned(to_signed(-2, pwm_cnt'
           length)) then
          pwm_cnt <= (others => '0');
        end if;

        if pwm_cnt < duty_cycle then
          pwm_out <= '1';
```

```
18          end if;
19        end if;
20      end if;
21    end if;
22  end process;
```

Listing 4: Proceso de generación de PWM

En este bloque se observa:

1. Reinicio del contador y salida cuando `rst` está activo.
2. Incremento del contador `pwm_cnt` en cada ciclo válido.
3. Comparación con el valor de `duty_cycle` para encender o apagar la salida.

### Proceso de División de Reloj

El proceso `CLK_CNT_PROC` se utiliza como prescaler. Esto permite ajustar la frecuencia del PWM sin modificar la frecuencia de reloj principal.

```
1  CLK_CNT_PROC : process( clk )
2  begin
3    if rising_edge(clk) then
4      if rst = '1' then
5        clk_cnt <= 0;
6      else
7        if clk_cnt < clk_cnt_len - 1 then
8          clk_cnt <= clk_cnt + 1;
9        else
10         clk_cnt <= 0;
11       end if;
12     end if;
13   end if;
14 end process;
```

Listing 5: Proceso de división de reloj

Este prescaler es especialmente útil para evitar que la señal PWM tenga una frecuencia demasiado alta, lo que podría impedir la correcta percepción del cambio de brillo en el LED.

## 2.2. Modulo 2: Circuito Contador

Este módulo implementa un contador genérico que incrementa su valor en cada flanco ascendente del reloj cuando la señal `count_enable` está activa. El contador es auto envolvente, lo que significa que cuando alcanza su valor máximo, vuelve a cero automáticamente. El rango máximo depende del número de bits especificado por el parámetro genérico.

### 2.2.1. Código VHDL Completo del Contador

```
1  library ieee;
2  use ieee.std_logic_1164.all;
3  use ieee.numeric_std.all;
4  use ieee.math_real.all;
5
6  entity counter is
7    generic (
8      counter_bits : integer
9    );
10   port (
11     clk          : in std_logic;
12     rst          : in std_logic;
13     count_enable : in std_logic;
14     counter      : out unsigned (counter_bits - 1 downto
           0)
15   );
16 end counter;
17
18 architecture rtl of counter is
19   signal counter_i : unsigned(counter'range);
20 begin
21
22   counter <= counter_i;
23
24   COUNTER_PROC : process(clk)
25   begin
26     if rising_edge(clk) then
27       if rst = '1' then
28         counter_i <= (others => '0');
29       else
30         if count_enable = '1' then
31           counter_i <= counter_i + 1;
32         end if ;
```

```
33        end if ;
34      end if ;
35    end process;
36  end architecture;
```

Listing 6: Código completo del módulo Contador

### 2.2.2. Definición de la Entidad y Parámetro Genérico

```
1  entity counter is
2    generic (
3      counter_bits : integer
4    );
5    port (
6      clk          : in std_logic;
7      rst          : in std_logic;
8      count_enable : in std_logic;
9      counter      : out unsigned (counter_bits - 1 downto
           0)
10   );
11 end counter;
```

Listing 7: Entidad del módulo Contador

En esta entidad se define:

- **counter_bits**: parámetro genérico que establece el número de bits del contador, determinando su rango máximo.
- **clk**: señal de reloj.
- **rst**: señal de reinicio síncrono que pone el contador en cero.
- **count_enable**: habilitador que permite o detiene la cuenta.
- **counter**: salida del valor actual del contador.

### 2.2.3. Señales Internas

Dentro de la arquitectura se define una señal interna `counter_i`, que almacena el valor actual del contador y se asigna continuamente a la salida `counter`.

```
1  signal counter_i : unsigned(counter'range);
```

Listing 8: Definición de señal interna del contador

### 2.2.4. Proceso de Conteo

El proceso `COUNTER_PROC` se activa en cada flanco ascendente del reloj. Si la señal `rst` está activa, el contador se reinicia a cero. Si `count_enable` está en alto, el valor del contador se incrementa en una unidad.

```
COUNTER_PROC : process(clk)
begin
  if rising_edge(clk) then
    if rst = '1' then
      counter_i <= (others => '0');
    else
      if count_enable = '1' then
        counter_i <= counter_i + 1;
      end if ;
    end if ;
  end if ;
end process;
```

Listing 9: Proceso de conteo

Este mecanismo permite implementar fácilmente contadores de diferente tamaño ajustando únicamente el parámetro `counter_bits`, sin modificar el resto del código.

## 2.3. Modulo 3: Circuito Reset

Este módulo implementa un circuito de *reset* sincronizado y filtrado, encargado de generar una señal de reinicio interno (`rst`) a partir de una señal externa (`rst_n`). Su función es garantizar que el reinicio del sistema ocurra de forma controlada, evitando activaciones indeseadas provocadas por rebotes o inestabilidades en la señal externa.

### 2.3.1. Código VHDL Completo del Circuito Reset

```vhdl
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity reset is
  port (
    clk : in std_logic;
    rst_n : in std_logic;
    rst : out std_logic
  );
end reset;

architecture rtl of reset is

  signal sreg : std_logic_vector(3 downto 0);

begin
  SREG_PROC : process(clk)
  begin
    if rising_edge(clk) then
      sreg <= sreg(sreg'high - 1 downto 0) & rst_n;
    end if ;
  end process;

  RESET_PROC : process(sreg)
    constant all_ones : std_logic_vector(sreg'range) :=
       (others => '1');
  begin
    if sreg = all_ones then
      rst <= '0';
    else
      rst <= '1';
    end if ;
  end process;

end architecture;
```

Listing 10: Código completo del módulo Reset

### 2.3.2. Definición de la Entidad

```
entity reset is
  port (
    clk : in std_logic;
    rst_n : in std_logic;
    rst : out std_logic
  );
end reset;
```

Listing 11: Entidad del módulo Reset

En esta entidad:

- **clk**: señal de reloj utilizada para sincronizar el filtrado del reset.
- **rst_n**: señal externa de reset, activa en bajo.
- **rst**: señal de reset interno, activa en alto.

### 2.3.3. Señales Internas

El módulo emplea un registro de desplazamiento (`sreg`) de 4 bits para filtrar y estabilizar la señal de entrada. Esto permite que la señal de reset interno solo se desactive cuando la entrada externa se haya mantenido estable por varios ciclos de reloj.

```
signal sreg : std_logic_vector(3 downto 0);
```

Listing 12: Registro de desplazamiento para filtrado

### 2.3.4. Proceso de Registro de Desplazamiento

El proceso `SREG_PROC` captura el estado de la señal `rst_n` en cada flanco ascendente del reloj y lo va desplazando dentro del registro `sreg`. Esto actúa como un filtro contra cambios rápidos o inestables.

```
SREG_PROC : process(clk)
begin
  if rising_edge(clk) then
    sreg <= sreg(sreg'high - 1 downto 0) & rst_n;
  end if ;
end process;
```

Listing 13: Proceso de desplazamiento de la señal rst_n

### 2.3.5. Proceso de Generación de la Señal Reset Interna

El proceso `RESET_PROC` compara el contenido del registro `sreg` con un vector de todos unos (`all_ones`).

- Si todos los bits son '1', significa que la señal externa `rst_n` ha estado en alto de forma estable, y por lo tanto se libera el reset interno (`rst` = '0').
- En cualquier otro caso, el reset interno permanece activo (`rst` = '1').

```
RESET_PROC : process(sreg)
  constant all_ones : std_logic_vector(sreg'range) := (
     others => '1');
begin
  if sreg = all_ones then
    rst <= '0';
  else
    rst <= '1';
  end if ;
end process;
```

Listing 14: Proceso de generación del reset interno

## 2.4. Archivo principal

En este módulo de nivel superior se integran los tres bloques: `PWM`, `Contador` y `Reset`. La idea es usar una onda de sierra (producida por el contador) para formar el *duty cycle* del PWM y, con ello, variar el brillo de un LED. La señal `count_enable` del contador se fija en `'1'` para evitar errores de uso y asegurar el conteo continuo.

### 2.4.1. Código VHDL completo del nivel superior

```vhdl
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

-- Entidad principal: integra PWM, contador y circuito de reset
entity pwm_led is
  generic(
    -- Resolucion del PWM en bits; define los niveles de duty cycle
    pwm_bits : integer := 8;

    -- Prescaler interno del PWM (divide el reloj)
    -- Frecuencia del PWM (aprox):
    --   f_pwm = f_clk / ((2^pwm_bits - 1) * clk_cnt_len)
    -- Ejemplo con f_clk = 100 MHz:
    --   f_pwm = 100e6 / ( (255) * 392 ) = 1000.4 Hz
    clk_cnt_len : positive := 392;

    -- Ancho del contador que genera la onda de sierra (duty base)
    -- Frecuencia de la sierra (envolvente):
    --   f_saw = f_clk / 2^cnt_bits
    -- Para f_clk = 100 MHz y cnt_bits = 28:
    --   f_saw = 100e6 / 268,435,456 = 0.37 Hz (periodo = 2.68 s)
    -- Ojo: a veces se escribe al reves; esta es la relacion correcta.
    cnt_bits : integer := 28
  );
  port (
    clk   : in std_logic;  -- Reloj del sistema
    rst_n : in std_logic;  -- Reset externo activo en bajo (pull-up)
    led   : out std_logic  -- Salida al LED
  );
end pwm_led;

architecture rtl of pwm_led is
  -- Senales internas
```

```
35    signal rst      : std_logic;
                                    -- Reset sincronizado/
          filtrado
36    signal cnt      : unsigned (cnt_bits - 1 downto 0);
              -- Contador principal (sierra)
37    signal pwm_out : std_logic;
                                    -- Salida PWM
38
39    -- Alias: toma los 'pwm_bits' mas significativos de '
          cnt' como duty_cycle del PWM.
40    -- Esto produce una rampa de duty entre 0 y (2^
          pwm_bits - 1) que se repite
41    -- a la velocidad de la sierra (f_saw).
42    alias duty_cycle is cnt(cnt'high downto cnt'length -
          pwm_bits);
43
44  begin
45    -- El LED sigue directamente la salida del PWM
46    led <= pwm_out;
47
48    -- 1) Reset: sincroniza y filtra rst_n para generar '
          rst' activo en alto
49    RESET : entity work.reset(rtl)
50      port map(
51        clk   => clk,
52        rst_n => rst_n,
53        rst   => rst
54      );
55
56    -- 2) PWM: genera la senal PWM con duty definido por
          los bits altos de 'cnt'
57    PWM : entity work.pwm(rtl)
58      generic map (
59        pwm_bits    => pwm_bits,
60        clk_cnt_len => clk_cnt_len
61      )
62      port map (
63        clk        => clk,
64        rst        => rst,
65        duty_cycle => duty_cycle,
66        pwm_out    => pwm_out
67      );
68
```

```
69    -- 3) Contador: produce la sierra. Habilitado
          permanentemente (count_enable = '1')
70    COUNTER : entity work.counter(rtl)
71      generic map (
72        counter_bits => cnt'length
73      )
74      port map (
75        clk          => clk,
76        rst          => rst,
77        count_enable => '1',  -- Conteo continuo (evita
              errores de usuario)
78        counter      => cnt
79      );
80  end architecture;
```

Listing 15: Archivo principal: integración de PWM, contador y reset

### 2.4.2. Análisis de parámetros y diseño

#### Genéricos del nivel superior

- `pwm_bits`: resolución del *duty*. A mayor valor, transición de brillo más suave.
- `clk_cnt_len`: prescaler del PWM. Aumentarlo reduce la frecuencia del PWM (útil para no exceder la respuesta de la carga).
- `cnt_bits`: controla la velocidad de la rampa (sierra). Con $f_{\text{clk}} = 100\,\text{MHz}$, $f_{\text{saw}} = f_{\text{clk}}/2^{\text{cnt_bits}}$.

#### Señales internas y alias

- `cnt`: contador libre que da la forma de onda base (sierra).
- `duty_cycle` (alias): toma los bits más altos de `cnt` para generar un duty proporcional al valor instantáneo de la sierra.
- `rst`: reset interno sincronizado, activo en alto, derivado de `rst_n`.

#### Flujo de datos

1. `counter` incrementa `cnt` (sierra).
2. El alias `duty_cycle` extrae los `pwm_bits` más altos de `cnt`.

3. `pwm` compara `pwm_cnt` vs `duty_cycle` y produce `pwm_out`.

4. `led` sigue `pwm_out`.

# 3. Diagrama de Bloques

En esta sección se presenta el diagrama de bloques del módulo principal `pwm_led` y los submódulos que lo componen. Cada bloque representa una entidad VHDL independiente que cumple una función específica dentro del sistema general.

## 3.1. Diagrama General del Módulo Principal

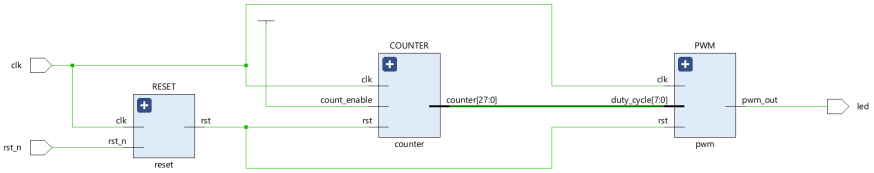



Figura 1: Diagrama de bloques del módulo principal `pwm_led`.

El módulo principal integra tres submódulos:

- **RESET**: Se encarga de generar una señal de reinicio controlada a partir de la entrada `rst_n`.
- **COUNTER**: Genera una onda de sierra mediante un contador binario ascendente.
- **PWM**: Convierte la señal de onda de sierra en una señal de modulación por ancho de pulso (PWM) para controlar el brillo del LED.

## 3.2. Módulo RESET

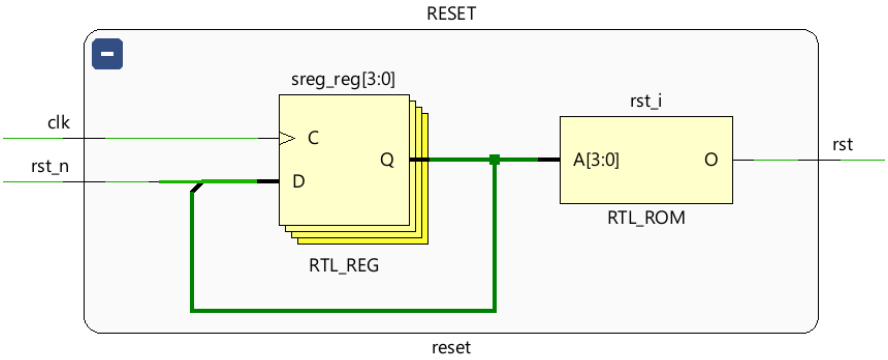



Figura 2: Bloque funcional del módulo `RESET`.

Este módulo recibe la señal de reloj `clk` y la señal de reinicio activa en bajo `rst_n`. Internamente utiliza un registro de desplazamiento para filtrar rebotes o ruidos, asegurando que la salida `rst` sea estable antes de iniciar el resto del sistema.

## 3.3. Módulo COUNTER

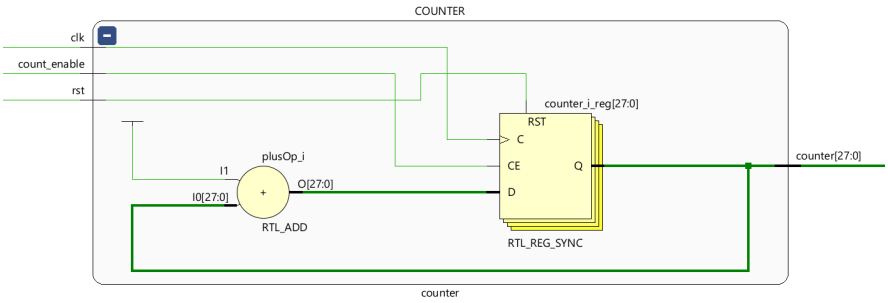



Figura 3: Bloque funcional del módulo `COUNTER`.

Este módulo es un contador binario ascendente que genera una onda de sierra digital. Su frecuencia de incremento está determinada por la señal de reloj `clk` y el número de bits de conteo. La salida `cnt` se usa para determinar el ciclo de trabajo (`duty_cycle`) del PWM.

### 3.4. Módulo PWM

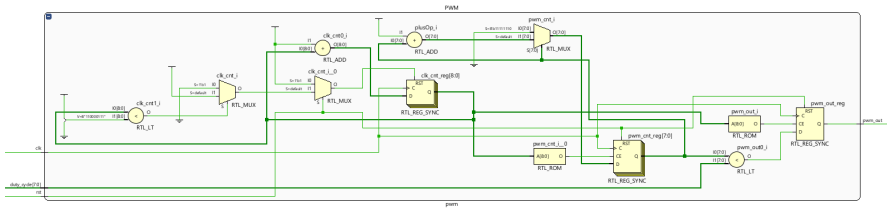


Figura 4: Bloque funcional del módulo `PWM`.

Este módulo compara el valor del `duty_cycle` con un contador interno para generar una señal PWM. Cuando el contador es menor que el ciclo de trabajo, la salida `pwm_out` se mantiene en alto; de lo contrario, se pone en bajo. Esto permite controlar el brillo del LED dependiendo del tiempo que la señal se mantenga activa en cada ciclo.

## 4. Asignación de Pines en la Nexys A7-100T

Para la implementación en la tarjeta **Nexys A7-100T** se requiere definir las conexiones físicas entre las señales del diseño VHDL y los pines del FPGA. La **asignación de pines** se realiza en el archivo de restricciones `.xdc` y permite garantizar que cada señal se conecte al componente físico correcto de la placa.

En este proyecto se utilizan únicamente tres señales: el reloj (`CLK`), la señal de reinicio activo bajo (`RST_n`) y la salida hacia el LED (`LED_OUT`).

| Señal | Pin FPGA | Descripción |
|---|---|---|
| CLK | E3 | Entrada de reloj principal de 100 MHz que provee la Nexys A7. |
| RST_n | J15 | Pulsador de usuario conectado con lógica activa en bajo, usado para reiniciar el sistema. |
| LED_OUT | H17 | LED de usuario en la parte superior derecha, usado para visualizar la señal PWM generada. |

Cuadro 1: Asignación de pines en la Nexys A7-100T.

### Justificación de la selección de pines

- **CLK (E3):** Este pin está cableado internamente al oscilador de 100 MHz de la placa, por lo que es la opción estándar para el reloj del sistema.
- **RST_n (J15):** Se elige este pin porque corresponde a uno de los switch de usuario (SW0).
- **LED_OUT (H17):** Este pin controla uno de los LEDs de usuario, permitiendo observar visualmente el resultado del PWM en tiempo real.

## 5. Resultados en la Placa Nexys A7 100T

Para validar el funcionamiento del diseño implementado en la **Nexys A7-100T**, se realizaron pruebas prácticas observando el comportamiento del LED de salida (LED_OUT) al cambiar el estado del interruptor asociado a la señal RST_n.

### Caso 1: Switch en bajo (sin salida)

En este estado, el interruptor de entrada se mantiene en posición baja. El sistema permanece en reposo y el LED LED_OUT se mantiene apagado.

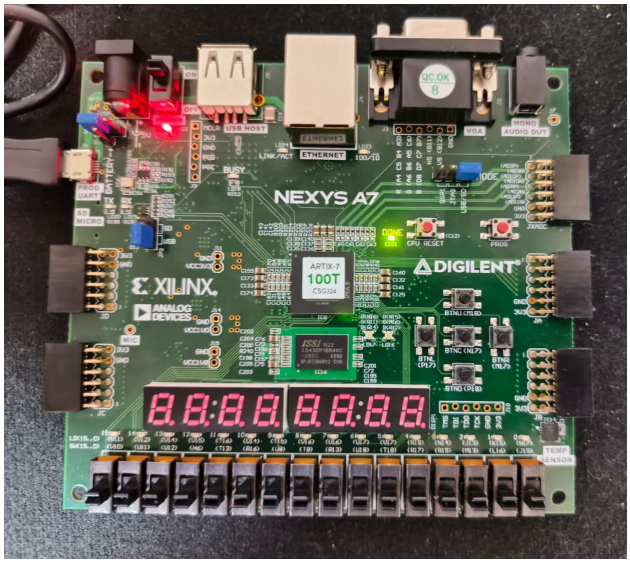



Figura 5: Switch en bajo: sin señal de salida, LED apagado.

## Caso 2: Switch en alto (LED encendido)

Cuando el interruptor pasa a estado alto, el sistema activa la salida y el LED correspondiente se enciende, confirmando el funcionamiento correcto de la lógica implementada.

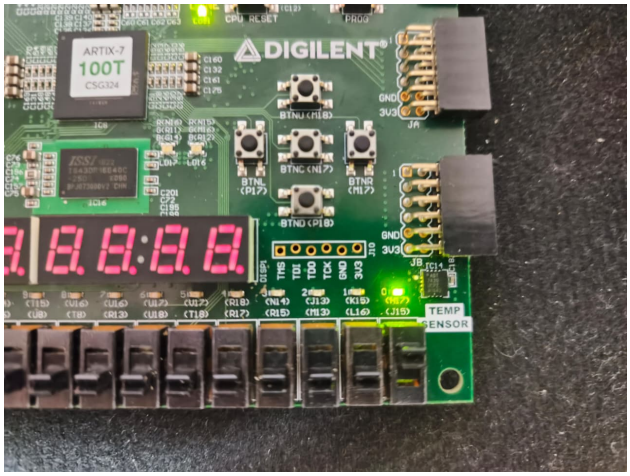



Figura 6: Switch en alto: salida activa, LED encendido.